

Reinforcement-Learning-Based Control for NO_x Emission Reduction in a Hybrid Diesel Vehicle

Niklas Long Schiefelbein

Master's Thesis Defence
Supervisor: Cecilio Angulo Bahón

IDEAI – Universitat Politècnica de Catalunya
within the LDCI2027 / CORNET project

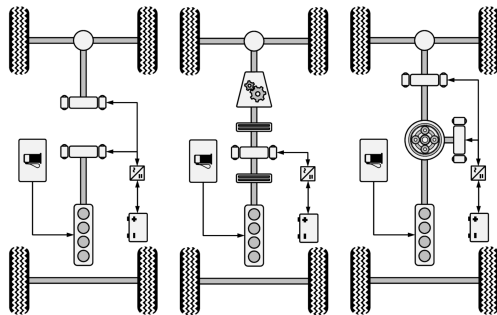
2026

- 1 Motivation & Problem
- 2 Approach
- 3 Phase 1 – Feasibility & the Twist
- 4 Phase 2 – Multi-Objective Control
- 5 Generalisation: WLTC
- 6 Conclusion

Why this matters: clean hybrid powertrains

- Transport $\approx$ **one third** of EU greenhouse-gas emissions.
- Combustion cars still $\approx 31\%$ of new sales $\Rightarrow$ improving them still matters.
- **Hybrid Electric Vehicles (HEV)** = bridge: combustion range + electric efficiency.
- Our target pollutant: **NO_x** (nitrogen oxides) – a health hazard, capped by **Euro 6 at 80 mg/km**.

Goal of this work: make an HEV emit *less* NO_x without breaking anything else.



Powertrain architectures.

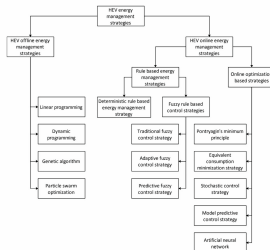
The control problem: four goals fighting at once

An HEV controller must **simultaneously**:

- 1 follow the **driver's speed**
- 2 cut **NO_x** emissions
- 3 keep the **battery charged**
(charge-sustaining)
- 4 decide in **< 1 ms** (real time)

Every classical method breaks somewhere:

- **Dynamic Programming** – optimal, but needs the *whole future* + too slow.
- **Rule-based** – reliable, but hand-tuned per vehicle.
- **MPC / ECMS** – need forecasts / constant retuning.



Why Reinforcement Learning – and where this thesis sits

RL promises to handle all four jointly:

- learns **offline**, runs **instantly** (one forward pass)
- **no hand-written rules** – learns from interaction
- naturally **multi-objective** via the reward

Project context (LDCI2027):

- TU Berlin = engine test bench, IFS = plant models, **UPC = control (this work)**.

Honest starting point:

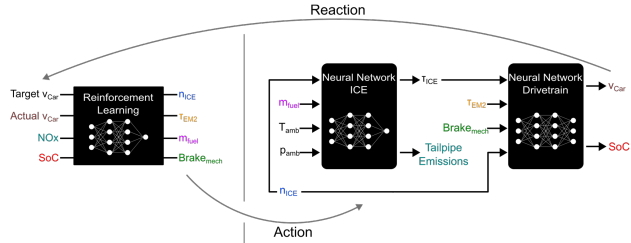
Prior project work could not even *stabilise* the speed.

This thesis **steps back** to get something that works:

- non-recurrent agents
- approximate the problem as an **MDP**
- track a **full trajectory**, not one fixed speed

Training environment: a fast LSTM surrogate

- Physical engine simulation is **too slow** for millions of RL steps.
- Replace it with two cascaded **LSTM** surrogate models:
 - ICE** surrogate $\rightarrow$ torque + NO_x
 - drivetrain** surrogate $\rightarrow$ speed + battery state
- Closed loop: agent acts $\rightarrow$ surrogate predicts $\rightarrow$ reward $\rightarrow$ repeat.



What the agent sees, does, and is rewarded for

Observes (measurable only)

- speed + tracking error
- battery state (SOC)
- engine state, fuel, NO_x
- 1 thermal proxy (SCR temp)
- last actions

Controls

- engine on/off + rpm
- electric-motor torque
- fuel injected
- mechanical brake

Reward

- core: **speed tracking** (peak at zero error)
- penalties: NO_x , SOC drift
- small shaping: braking, engine flicker

Key principle

The reward is only a **proxy**. Agents are selected on the **true measured objective** (speed RMSE), *not* on accumulated reward.

The core method idea: a two-phase curriculum

Don't learn everything at once – learn to *walk* before you optimise.

Phase 1 – Feasibility

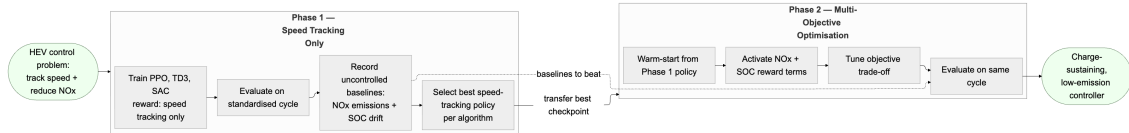
Reward = **speed tracking only**.

Question: can RL drive the car at all? What side-effects appear?

Phase 2 – Multi-objective

Warm-start from Phase 1, then **add NO_x + SOC** penalties.

Question: can we get clean + charge-sustaining *without losing tracking*?



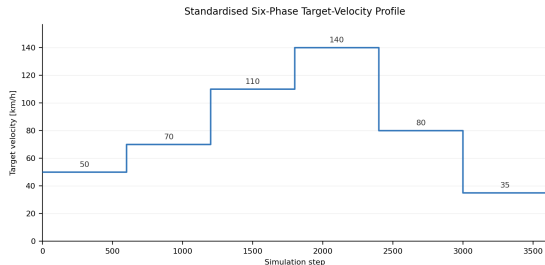
Three actor-critic algorithms

- **PPO** – simple, on-policy baseline (sanity check)
- **SAC** – off-policy, sample-efficient
- **TD3** – off-policy, used in related HEV work

Hyperparameters chosen with **Optuna**; each config re-trained over **10 seeds**.

Evaluation

- deterministic **staircase** speed cycle
- primary metric: speed **RMSE**
- diagnostics: cumulative NO_x , fuel, ΔSOC
- report mean $\pm$ std over 10 seeds



All three algorithms track the speed cycle well – **feasibility proven**.

Metric (eval cycle)	PPO	TD3	SAC
RMSE speed [km/h]	4.19 ± 0.98	3.81 ± 0.29	3.45 ± 0.43
Cumulative NO _x [g]	48.3	95.4	63.8
Δ SOC	+0.25	+0.15	+0.19

All under 5 km/h RMSE. **But the NO_x and battery numbers hide something...**

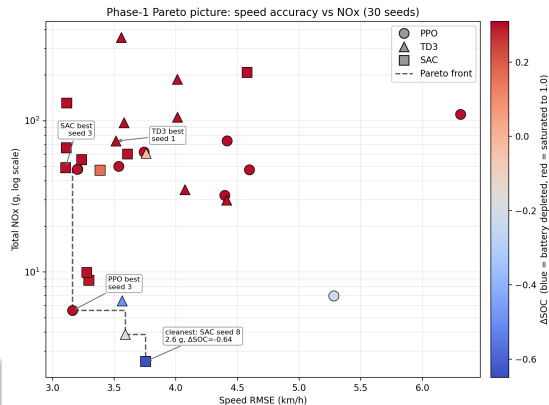
The twist: equal tracking, very different behaviour

Looking clean is easy if you cheat.

- Same speed reward $\rightarrow$ wildly different powertrain use.
- The “cleanest” Phase 1 seed (SAC, 2.6 g NO_x) got there by draining the battery ($\Delta\text{SOC} = -0.64$).
- Only 1 of 30 seeds was charge-sustaining – and it emitted 60.9 g NO_x .

Lesson

Low NO_x is **meaningless** without a **charge-sustaining constraint**. $\Rightarrow$ motivates Phase 2.



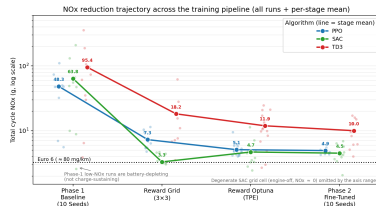
Colour = ΔSOC . Low- NO_x front = battery-depleters.

Phase 2: fix it with NO_x + SOC penalties

Fine-tune from Phase 1; activate NO_x and SOC reward terms.

Metric (P1 → P2)	PPO	TD3 (8/10)	SAC
RMSE [km/h]	4.19 → 3.13	3.81 → 3.96	3.45 → 3.33
NO _x [g]	48.3 → 4.9	95.4 → 8.7	63.8 → 4.5
Fuel [g]	79.6 → 52.8	122.6 → 79.5	113.5 → 64.0
ΔSOC	0.25 → 0.03	0.15 → 0.07	0.19 → 0.01

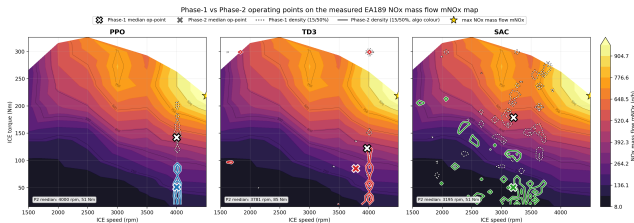
NO_x down $\sim 10\times$, SOC drift down $\sim 10\times$, tracking kept (even improved).



Why it works – not a black box

- NO_x rises **monotonically** with engine torque ($r = 0.84$).
- Phase 2 shifts the engine to **lower load** $\rightarrow$ lower NO_x .
- Fuel drops *too* – but only because Phase 1 was **overcharging** the battery with wasteful high-load running.

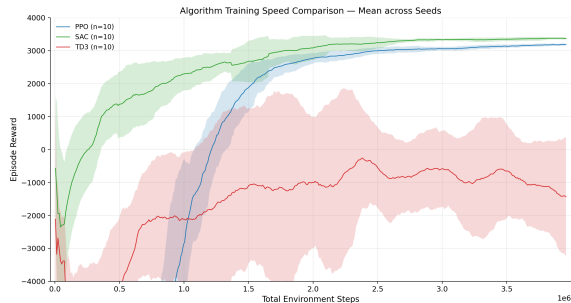
Not a general NO_x –fuel synergy – it's removing wasted work.



Operating points move from bright (high- NO_x) into dark (low- NO_x) regions.

Algorithm verdict

- **PPO** – most **robust**: tiny seed-to-seed variance, best tracking transfer.
- **SAC** – **cleanest**: lowest NO_x , best single candidate.
- **TD3** – **excluded**: unstable, 20 % of seeds fail to converge.



TD3 (red) never stabilises; PPO/SAC converge cleanly.

Generalisation test: zero-shot on the real WLTC cycle

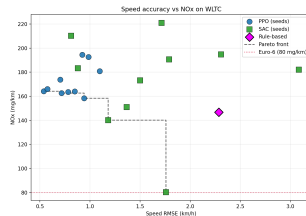
WLTC = standardised real-world driving cycle. Agents **never trained on it**.

The honest population-level result

Averaged over 10 seeds, the RL agents **lose** to the rule-based controller:

- burn **1.6–2.2× more fuel**, emit **~17 % more NO_x**
- **but** track the target more tightly (RMSE 0.8–1.6 vs 2.3 km/h)

Reporting this loss is deliberate: fuel was *never* in the reward – a known blind spot.

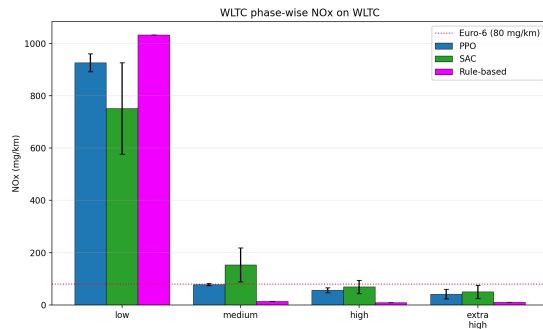


But the right unit is a single controller, not an average

Averaging distinct policies describes no real controller. Pick the best one.

	NO _x mg/km	Fuel g	RMSE km/h	Δ SOC
SAC seed 7	80	1730	1.76	+0.01
Rule-based	147	1376	2.29	+0.00

SAC seed 7 hits Euro 6 (80 vs 147 mg/km),
tracks better, stays charge-neutral.
Cost: +26 % fuel, achieved by switching the engine off
57 % of the cycle.



- **Single pollutant.** Only NO_x optimised – no fuel/CO₂ term. *The* main gap.
- **Surrogate only.** Validated against LSTM models, not a real engine – sim-to-real open.
- **Engine-specific.** Models tied to the EA189; the *methodology* transfers, the models don't.
- **Asymmetric baseline.** Rule-based = 1 run; RL = 10 seeds. Informative, not a strict superiority test.

This is a proof of concept – feasibility shown, deployment not claimed.

Contributions

- surrogate-based DRL control framework
- curriculum-inspired two-phase training
- comparative PPO / SAC / TD3 study
- zero-shot WLTC generalisation test
- design guidance for the LDCI2027 project

Headline

- NO_x cut $\sim 10\times$, charge-sustaining, tracking kept
- **SAC seed 7** reaches Euro 6 zero-shot $\rightarrow$ handed to LDCI2027

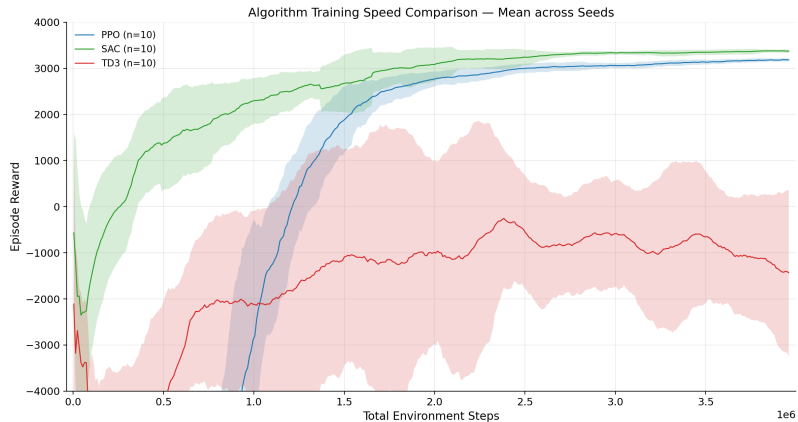
Future work

- add fuel / CO_2 reward term
- recurrent agents (true POMDP)
- hybrid rule-based + RL supervisor

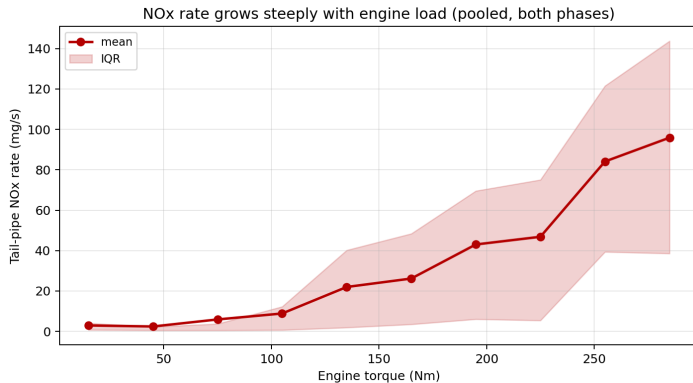
Thank you

Questions?

Backup: full Phase 2 training curves

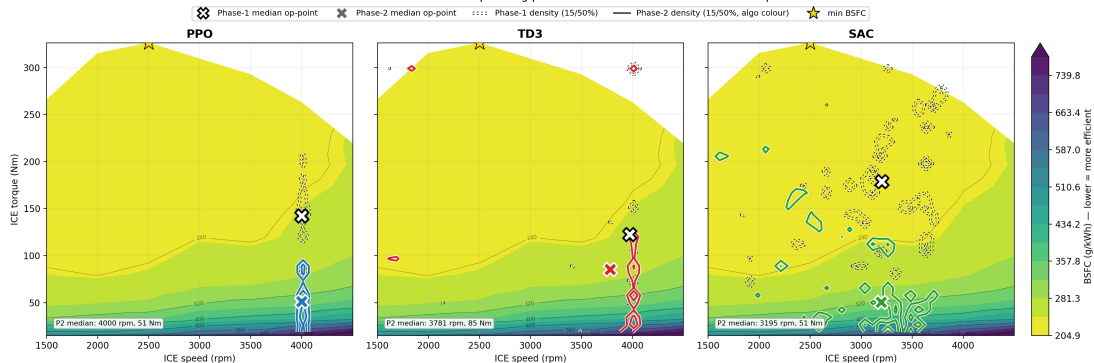


Backup: NO_x rate vs engine torque ($r = 0.84$)

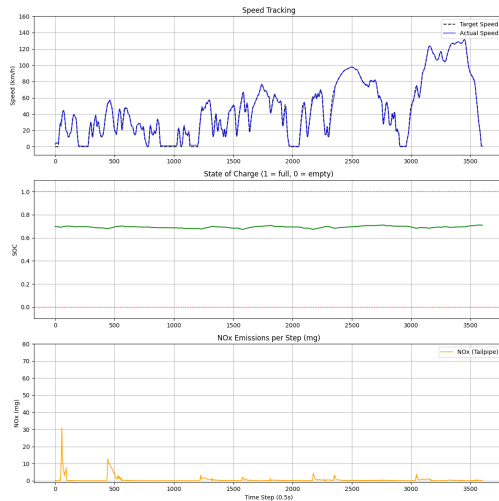


Backup: BSFC engine-map operating points (P1 vs P2)

Phase-1 vs Phase-2 operating points on the measured EA189 BSFC map



Backup: SAC seed 7 on WLTC – full trace



Backup: WLTC speed tracking (10-seed band)

